CSE3069 · PHÁT TRIỂN ỨNG DỤNG IOT — BÀI THỰC HÀNH 04 / 10

Gửi dữ liệu qua MQTT

Dựng broker và nhận số đo thật từ ESP32 — firmware tách nhiều tác vụ, không bị chặn khi mạng chậm.

TUẦN	HÌNH THỨC	MÔI TRƯỜNG	THỜI LƯỢNG GỢI Ý
09	Cá nhân	Mô hình nhà kính + broker trên PC/VPS nhóm	150–180 phút

1 YÊU CẦU

Dựng một **broker MQTT** trên PC, Raspberry Pi hoặc VPS của nhóm. Cho ESP32 trên **mô hình nhà kính** gửi số đo thật (nhiệt độ – độ ẩm, cảm biến mưa) tới broker; dùng một máy khác để nhận và đối chiếu dữ liệu. Firmware phải **tách thành nhiều tác vụ** (đọc cảm biến / gửi MQTT / giữ an toàn) để mạng chậm không làm nghẽn toàn hệ.

2 CÁC BƯỚC THỰC HIỆN

1 Dựng broker bằng Docker Compose

Đưa cấu hình broker (Mosquitto hoặc EMQX) vào Docker Compose, cấu hình tài khoản và kết nối trong môi trường lab.

LÀM GÌ	Viết `docker-compose.yml` với service broker, mount file cấu hình tài khoản (`password_file`) và mở cổng 1883.
KHÔNG ĐƯỢC LÀM	**Không đưa mật khẩu plaintext vào repo** — dùng biến môi trường hoặc file cấu hình nằm ngoài git (thêm vào `.gitignore`).
KIỂM TRA	Chạy `docker compose up -d`, dùng `mosquitto_sub/mosquitto_pub` hoặc MQTT Explorer từ một máy khác để xác nhận broker nhận/gửi được.

2 Thống nhất topic và payload

Thiết kế cấu trúc topic và nội dung bản tin gồm thiết bị, thời gian, số đo/đơn vị và cờ chất lượng.

MẪU TOPIC	`nhom01/node01/telemetry` — đặt tên nhóm và mã node rõ ràng, không dùng tên chung chung như `test`.

MẪU PAYLOAD	`{"device_id":"node01","ts":123456,"temp":27.4,"unit":"C","valid":true}` — tái dùng cách phân biệt `null`/giá trị lỗi đã làm ở Bài 2.

3 Publish và kiểm tra liên tục 10 phút

Cho ESP32 publish; từ máy khác subscribe đúng topic. Lưu log nhận liên tục ít nhất 10 phút.

LÀM GÌ	Chạy `mosquitto_sub -t "nhom01/node01/telemetry" -v | tee log.txt` trên máy quan sát, để chạy song song trong lúc ESP32 publish.
KIỂM TRA	Đối chiếu số lượng bản tin nhận được với chu kỳ gửi đã đặt — có bị rớt bản tin nào không, nếu có thì ghi lại thời điểm.

4 Thử ngắt kết nối broker có chủ đích

Ngắt kết nối broker có chủ đích rồi nối lại. Kiểm node phục hồi kết nối và vòng điều khiển an toàn vẫn hoạt động.

LÀM GÌ	Dừng container broker (`docker compose stop broker`) trong 30 giây rồi khởi động lại.
QUAN SÁT GÌ	Firmware có tự kết nối lại khi broker sống lại không? Trong lúc mất kết nối, tác vụ an toàn (FSM Bài 3) có còn phản hồi được không?

> ⚠ **Bẫy thường gặp**
> Nếu gộp hết việc đọc cảm biến, gửi MQTT và kiểm tra an toàn vào một `loop()`, một lần gọi MQTT bị treo (do mất mạng) sẽ làm toàn bộ chương trình đứng theo — kể cả phần an toàn.

3 THÍ NGHIỆM BẮT BUỘC — FREERTOS, TASK VÀ QUEUE

Tách firmware thành **ít nhất hai task** chạy song song (ví dụ: task đọc cảm biến + gửi MQTT, và task giữ an toàn), trao đổi dữ liệu qua **queue có giới hạn**.

CẤU TRÚC	Dùng `xTaskCreate()` tạo task riêng cho MQTT và task an toàn với chu kỳ kiểm tra riêng không phụ thuộc MQTT. Hàm này không tự ghim lõi; nếu cần chỉ định core trên chip hỗ trợ, dùng `xTaskCreatePinnedToCore()`.
QUEUE	Dùng `xQueueCreate()` với kích thước cố định (vd. 10 bản ghi). Quyết định chính sách khi đầy: chờ, bỏ bản ghi cũ nhất, hay ghi đè — ghi rõ lựa chọn.
THỬ TẢI	Làm bên gửi MQTT chậm hơn bên đọc (ví dụ: chèn `vTaskDelay()` dài trong task MQTT) rồi quan sát queue đầy dần — ghi số bản ghi bị bỏ/giữ và xác nhận task an toàn vẫn chạy đúng nhịp.

nộp: sơ đồ task/queue | firmware FreeRTOS | log thí nghiệm nghẽn queue

4 KẾT QUẢ CẦN ĐẠT

TIÊU CHÍ TỰ KIỂM

Tick khi đã hoàn thành — lưu lại trên máy này, không gửi đi đâu.

- [ ] Máy khác nhận được số đo thật từ ESP32 liên tục tối thiểu 10 phút.
- [ ] Payload đủ ngữ cảnh, phân biệt null và 0.
- [ ] Firmware tách ít nhất 2 task, có queue giới hạn rõ ràng.
- [ ] Ngắt broker có chủ đích, node tự kết nối lại, tác vụ an toàn không bị chặn.

`0 / 4 tiêu chí đã tự kiểm`

MÔI TRƯỜNG CHO PHÉP

ESP32 trên **mô hình nhà kính của nhóm** đọc cảm biến thật và publish. Broker MQTT chạy trên **PC, Raspberry Pi hoặc VPS** của nhóm; dùng một máy khác để subscribe và đối chiếu.

> ⚠ Chương trình Python trên PC có thể thử gửi/nhận MQTT trước để kiểm logic, nhưng **không thay** minh chứng FreeRTOS chạy trên ESP32 thật của mô hình nhà kính.

ĐỌC THÊM — 3 CẤP ĐỘ TRIỂN KHAI (NÊN THỬ CẢ BA ĐỂ CÓ CÁI NHÌN RỘNG)

Yêu cầu bài **giữ nguyên**: broker chạy trên PC/RPi/VPS của nhóm, ESP32 nhà kính publish số đo thật, firmware tách task FreeRTOS. Dưới đây là **ba cách triển khai cùng một luồng dữ liệu** — bắt đầu từ cấp 1 cho chắc, rồi leo dần. Hiểu được **vì sao đổi từ cấp này sang cấp kia** quan trọng hơn việc chạy được một cấp.

CẤP 1 · CƠ BẢN

ESP32 → Mosquitto local → máy nhận, trong cùng mạng LAN. Dựng Mosquitto bằng Docker Compose ngay trên PC của nhóm (cổng 1883). ESP32 trỏ `mqtt_server` về **IP LAN của PC** (vd `192.168.1.x`) — **không phải `localhost`**, vì với ESP32 `localhost` là chính nó. Nghiệm thu: mở `mosquitto_sub -t "nhom01/node01/telemetry" -v` trên máy khác, thấy số đo nhảy realtime. **Mục tiêu:** hiểu rõ broker chỉ trung chuyển — tắt ESP32 thì máy nhận im lặng, bật lại thì dữ liệu về tiếp.

Công cụ kiểm tra nhanh (không bắt buộc): ngoài `mosquitto_sub` trên dòng lệnh, có thể dùng một trang kiểm tra tối giản mở bằng trình duyệt để in thô mọi bản tin: **kiem-tra-mqtt.html ↗**. Trang này **chỉ để xác nhận broker/topic chạy đúng** — không phải giao diện nộp bài; phần thiết kế giao diện trực quan là việc nhóm em tự làm ở cấp 2.

CẤP 2 · NÂNG CAO

Tự dựng một bảng điều khiển trực quan làm subscriber thứ hai. Vẫn broker loca em **tự thiết kế giao diện** hiển thị realtime — chọn một trong hai hướng:

- **Node-RED** (service trong cùng Compose): node `mqtt in` → `function` kiểm ng Dashboard.
- **Tự viết HTML / CSS / JavaScript** (dùng thư viện MQTT over WebSocket, vd M hiển thị là **do nhóm em quyết định**.

Bổ sung bắt buộc — tầng an toàn, kế thừa TH2 & TH3:

- **Chất lượng dữ liệu (từ TH2):** hiển thị cờ `valid`; khi `valid:false` hoặc giá trị như số đo thật (phân biệt với giá trị `0` hợp lệ).
- **Machine state / máy trạng thái (từ TH3):** thể hiện trạng thái hệ **DỪNG · CHẠ** đưa về DỪNG (trạng thái an toàn).
- **Failsafe hiển thị:** báo trạng thái kết nối (đang kết nối / mất kết nối) và phát hiệ lâu — không để màn hình "đứng số cũ" đánh lừa người xem.

Gợi ý quy ước **chấm tròn trạng thái** cho giao diện:

● Kết nối OK · dữ liệu hợp lệ ● Đang nối lại · ngưng nhận ● Mất kết nối

Tùy chọn 2 chiều: nút trên giao diện publish `nhom01/node01/cmd` → ESP32 bật/tắt l

Điểm cần thấy: `mosquitto_sub` (cấp 1) và bảng điều khiển cùng subscribe một topi **subscriber**, mỗi bên một mục đích.

⚠ **Trình duyệt không kết nối thẳng cổng 1883.**
Cổng mặc định `1883` của MQTT chạy trên TCP thuần — TCP là giao thức tru JavaScript trong trang web không mở được kết nối TCP thuần; nó chỉ kết n kênh hai chiều trên nền web, cùng tầng với HTTP.
Vì vậy broker phải phục vụ thêm MQTT over WebSocket — tức MQTT đóng bằng cách khai báo thêm một `listener` (cổng lắng nghe) trong `mosquitt`

```
listener 1883          # ESP32 publish qua TCP (giữ nguyên)

listener 9001          # cổng cho trình duyệt

protocol websockets    # giao thức: WebSocket

allow_anonymous false  # dùng tài khoản như cấu hình broker ở bướ
password_file /mosquitto/config/passwords  # mount file tài khoản
```

Trong giao diện web điền địa chỉ dạng `ws://192.168.1.x:9001` (`ws://` = chạy broker). ESP32 vẫn publish qua cổng `1883` như cũ — hai đường cùng

CẤP 3 · CLOUD

Đổi broker sang cloud (HiveMQ Cloud / EMQX Cloud, bản free) — cấu hình lại kết nối TLS. Giữ nguyên logic ứng dụng và topic/payload; cấu hình `mqtt_server`, cổng **8883 (TLS)**, user/password và client TLS có kiểm chứng CA/tên máy chủ. Chỉ đổi cổng trên client TCP thường là chưa đủ. **Điểm cần thấy:** ứng dụng không đổi logic, chỉ đổi điểm kết nối — đây là tính **tách rời (decoupling)** của MQTT; và từ mạng 4G điện thoại vẫn subscribe được (điều local không làm được) → đó là lý do thật để lên cloud. **Bài học bảo mật:** public broker

	thì ai cũng đọc được topic → bắt buộc đặt username/password + TLS; thử sub topic bừa để tự thấy rủi ro.

Nộp bài vẫn theo yêu cầu gốc.
Ba cấp độ là để mở rộng hiểu biết. Nếu có làm cấp 2/cấp 3, ghi lại trong README (thêm ảnh Dashboard / cấu hình cloud) như phần nâng cao — không bắt buộc, nhưng được ghi nhận. Minh chứng bắt buộc vẫn là: log 10 phút, log ngắt/nối broker, và FreeRTOS chạy trên ESP32 thật.

5 KỸ NĂNG CẦN ĐẠT ĐƯỢC

D0	Thiết kế nơi triển khai (ESP32/PC/RPi) và thống nhất topic/payload trước khi viết code.
D3	Dựng broker bằng Docker — dựng lại được từ cấu hình repo.
C5 · C7	Truyền bản tin MQTT đúng topic đã thiết kế, cấu trúc nhất quán.
D1	FreeRTOS: tách task, dùng queue có giới hạn giữa các tác vụ.
D1 · D2	Thử queue đầy và mạng chậm — chứng minh vòng điều khiển an toàn không bị chặn.

6 BÀI NỘP

Nộp đúng MỘT thứ: link tới một kho Git (repo) của riêng em.
Toàn bộ sản phẩm bên dưới (Compose, firmware FreeRTOS, log, README trả lời câu hỏi) phải nằm trong repo đó theo đúng cấu trúc ở mẫu README. Dán link repo vào ô nộp của mục **TH04** trên SV20. Giảng viên chỉ chấm theo đúng nội dung đã commit trong repo — không nhận bài trả lời rời trên giấy, ảnh lẻ hay chat riêng.

NỘP THEO MSSV · CÁ NHÂN	1 LINK REPO CHỨA ĐỦ SẢN PHẨM
1	**Cấu hình broker + firmware + log** — `docker-compose.yml`, `firmware/`, `log_10min.txt`, `log_disconnect.txt`Compose đã bỏ bí mật + firmware FreeRTOS + log publish/subscribe ≥ 10 phút + log thử ngắt/nối broker. Phải commit vào repo.
2	**Sơ đồ task/queue và nơi triển khai** — `task_queue.png`, `topic_payload.md`Node/broker/service chạy ở đâu; task nào, queue giới hạn bao nhiêu. Vẽ tay chụp ảnh hoặc vẽ số đều được.

3 README.md trả lời 4 câu hỏi

— `README.md`Trả lời đủ 4 câu (4.1–4.4) ngay trong README của repo, mỗi câu một đoạn đầy đủ. Xem mẫu cấu trúc bên dưới.

MẪU CẤU TRÚC README.MD

```
# Bài 4 — Gửi dữ liệu qua MQTT

Họ tên: ... · MSSV: ... · Nhóm: ...



## File trong repo này

- docker-compose.yml — cấu hình broker (đã bỏ bí mật)

- firmware/ — code FreeRTOS cho ESP32

- topic_payload.md — mô tả topic và cấu trúc payload

- log_10min.txt — log subscribe liên tục 10 phút

- log_disconnect.txt — log thử ngắt/nối lại broker



## 4.1 Payload thực tế

...dán một payload thật, giải thích từng trường...



## 4.2 Task và queue

...task nào làm gì, queue giới hạn bao nhiêu, chính sách khi đầy...



## 4.3 Thí nghiệm ngắt kết nối

...thời điểm ngắt, bằng chứng tác vụ an toàn vẫn chạy, kết quả phục hồi...



## 4.4 Đóng góp cá nhân

...
```

4.1 Payload thực tế

Dán một payload từ node thật và giải thích từng trường. Payload cần định danh thiết bị, thời gian, số đo/đơn vị và cờ chất lượng — giá trị thiếu phải là `null`, không phải mẫu minh họa thay cho dữ liệu thật.

4.2 Task và queue

Các task trao đổi qua queue thế nào? Khi queue đầy thì chính sách là gì? Giải thích vì sao thời gian chờ của task an toàn không được phụ thuộc vô hạn vào MQTT.

4.3 Kết quả thí nghiệm ngắt kết nối

Ghi thời điểm ngắt broker, bằng chứng (log/heartbeat) cho thấy vòng an toàn vẫn chạy trong lúc mất kết nối, và kết quả sau khi phục hồi mạng.

4.4 Đóng góp cá nhân

Việc gì trong bài này em **tự làm**? Dẫn commit/log cụ thể làm minh chứng; nếu có phần được bạn hỗ trợ, ghi rõ phần đó và ai hỗ trợ.
